

ME 403 Project Assignment

This project focuses on utilizing the methods acquired through the semester on a simulated robotic arm. The arm is Unitree Z1 (an existing commercial robot arm). Its pro version is currently valued over 1 Million TL.

For this project, please refer to the folder uploaded on SUcourse along with the project document. The folder ("z1_files") involves the following codes:

- z1_torque.xml or z1_pos.xml (The file that describes the robot)
- scene_torque.xml or scene_pos.xml (The file that refers to the robot description and sets the scene)
- sample_z1_torque.py or sample_z1_pos.py (its name is self explanatory)

You are recommended to save this folder in your local drive and work in the same directory (add new files to that folder).

Z1 has some similarities to the robots utilized in the lab assignments. Basically, it has a base rotator, two more links that form an elbow, and three more tightly-packed joints towards its end-effector (forming a group, typically called as 'wrist' in the robotics field). It does not involve any parallel link structures. Each link's description -including the physical parameters such as its inertia, or dimensions, position of center-of-mass (COM), etc.- is provided in the file named: 'z1_torque.xml'. At the end of this document, you will see a section tailored for explaining some of the syntax seen in the XML file. That explanation is just an introductory text. Please analyze the provided files for further details.

You are provided with the simulation of the same robot in two different modes: torque and position modes. The torque mode can be practiced using the files at z1_files\z1_joint_torque_mode. When you run the python code 'sample_z1_torque.py', you must see the robot arm slowly collapsing to the ground. That's because there is no torque applied on its joints in the default setting. Gravity pulls all the bodies to the floor. The collapse happens gradually, because the model involves a heavy damping factor applied on all joints. You can see that setting in the XML file if you look carefully. On the right panel of MUJOCO's viewport, you will see drop-down menus named "Joint", "Control", and "Equality". Expand "Control" and see the sliders that allow you to set the torque on each joint. The robot would move according to your torque assignment online. Expand "Joint" and note that those values are in gray and they are passive. You are not allowed to set the values there directly.

In the position mode, the sliders under the "Control" panel directly assigns the rotation angle of the joints of the robot. The main project assignment will be executed in the torque mode. This sample setup is provided to allow you practice and check your own forward kinematics model. Feel free to play with that code and update it as you see fit.

Note that the sample codes involve several snippets that print what is in the scene. Please use them for your reference, feel free to delete them if you prefer. Apart from those echoes, you will see commands that construct the robot and a while loop that runs the simulation. Please do not alternate those pieces. Feel free to add new codes before the while loop and inside the while loop (example suitable areas are marked in the sample code).

Tasks

Please complete the set of tasks listed below:

Task 1: Please model the kinematics of the end-effector of the robot, using the homogeneous transformation matrices shown in the lectures. Your code should always refer to this derivation.

Task 2: Write a function that derives the Jacobian of a point on the body of the robot. This function should take the link index, and the position of the point with respect to the body frame of that link as inputs and make the due computation automatically, returning the Jacobian matrix.

Task 3: Calculate the generalized force of the gravitational forces applied on the structure. Then apply the due torques on the actuators to cancel the gravitational forces. Your calculations can be validated by checking if the robot keeps its initial configuration once the simulation begins. If the gravity is cancelled properly, the links should not move at all (in the torque mode, of course). Please validate your gravity cancellation with 20 different the initial configurations that span the workspace significantly.

Task 4: Obtain the generalized mass matrix using the Jacobian matrices derived for each link's COM. Then obtain the Coriolis matrix accordingly (using Christoffel Symbols, covered in the lectures). With these matrices computed, you are ready to implement system control.

Task 5: Derive a position controller that works in the configuration space. Use that controller to move the end-effector to various static points in the workspace. Your simulation should show the robot performing the due motion once it is started with any given valid initial and target configurations.

Task 6: Formulate a target configuration set as a function of time. The target configuration formula should be in the following format:

$$q_i = A_i \sin(w_i t)$$

where $i = 1, 2, 3$ and t represents the time parameter in seconds. You can adjust the amplitude (A_i) and the frequency (w_i) of these cyclic functions as you see fit. Please derive a position controller that works in the configuration space to make the robot follow the formulated motion.

Task 7: Derive an impedance controller for the end-effector. This impedance controller should drive the end-effector to a target static point regardless of the position of the target point.

Deliverables For The Report

Please prepare the following sections:

Task 1- Please show the derivation of the kinematic model on paper. Then explain how this model is coded. Finally, show an error plot that validates the kinematic model. This plot should show the mismatch between your code's estimated end-effector position and the end-effector position provided by the simulator. You can use the simulation in position mode for this step.

Task 2- Prepare an error plot that validates the calculated Jacobian matrices. Choose random initial configurations and calculate the miniscule movement of the COM of all links for a very small perturbation in the configuration vector. Compare the estimated movement of each COMs to those provided by the simulator. Please randomize the perturbed configuration variable index and repeat this check 20 times. Show the resulting errors in a table (20-by-n, where n stands for the number of links).

Task 3- Please prepare a code file that demonstrates the performance of the gravity compensation at any given configuration. The code should allow selecting different initial configurations and clearly indicate how the compensation is calculated. This step should be completed using the simulation in torque mode.

Task 4- Please provide the code that computes the generalized mass matrix and the Coriolis matrix. Briefly explain how this code works in your report.

Task 5- Derive your controller on paper. Show its performance at 3 random different initial/final configurations. For each one of those cases, please tune the controller parameters for two different objectives: i) Try minimizing the time it takes to reach the target, ii) Try minimizing the mean error magnitude through the motion. Show the resulting performances using plots that illustrate the configuration error vector magnitude vs time. This section will involve a total of 6 plots (two different objectives for a total of three cases).

Task 6- Show the formulation of your controller and provide the following two plots: i) a 3D plot that shows the target path vs the robot's end-effector's path, ii) The position error vs time for 1 minute of operation. Make sure this 1 minute spans at least two cycles of rotation of the links of the robot. Please choose the rotation frequencies accordingly.

Task 7- Please prepare a code file that demonstrates the performance of the impedance controller at arbitrary target end-effector positions. In demonstrating this task, your simulation should begin with an initial configuration that is a bit distant from the target configuration. The end-effector should converge to the target configuration under impedance force. Please comment on the resultant performances for various initial/target configurations.

Grading:

Task 1 & 2: 10 pt each

Task 3: 15 pt

Task 4: 5 pt

Task 5: 20 pt

Task 6: 20 pt

Task 7: 20 pt